

SQL NEW QUESTION 2

SQL Question: Event Average Finish Time

Marks: 20

Difficulty: Medium

Database: ATHLETICS_RESULTS_DB

Schema Details

1. Events Table		
Column Name	Data Type	Description
event_id	INT (PK)	Unique event ID
event_name	VARCHAR(100)	Name of the event
2. Athletes Table		
Column Name	Data Type	Description
athlete_id	INT (PK)	Unique athlete ID
athlete_name	VARCHAR(100)	Name of the athlete
3. Results Table		
Column Name	Data Type	Description
result_id	INT (PK)	Unique result ID
event_id	INT (FK)	Event in which the athlete participated
athlete_id	INT (FK)	Athlete who participated
finish_time	DECIMAL(5,2)	Finish time in seconds
status	ENUM('Recorded','Disqualified','DNS')	Status of the result

Sample Data

Events

event_id	event_name
1	100m Sprint
2	200m Sprint

Athletes

athlete_id	athlete_name
1	Alice
2	Brian
3	Carlos
4	Diana

Results

result_id	event_id	athlete_id	finish_time	status
201	1	1	11.20	Recorded
202	1	2	10.95	Recorded
203	1	3	12.10	Recorded
204	1	4	11.50	Disqualified
205	2	1	22.50	Recorded
206	2	2	21.80	Recorded
207	2	3	21.80	Recorded
208	2	4	23.10	DNS

Problem Statement

A national athletics federation stores athlete performance data for multiple track events.

Generate a report that, for every recorded result, displays:

- The event name
- The athlete name
- The athlete's finish time in seconds
- The average finish time of all recorded performances within the same event

Use the `AVG()` window function with `PARTITION BY` to calculate the average finish time separately for each event.

Only results where `status = 'Recorded'` should be included.

Results with statuses such as `Disqualified` or `DNS` must not appear in the final output and must not be included while calculating the event average.

Round the calculated average finish time to 2 decimal places.

Sort the final result by:

1. `event_name`
2. `finish_time`

3. athlete_name

Required Output Columns

- event_name
- athlete_name
- finish_time
- avg_event_time

Expected Output

event_name	athlete_name	finish_time	avg_event_time
100m Sprint	Brian	10.95	11.42
100m Sprint	Alice	11.20	11.42
100m Sprint	Carlos	12.10	11.42
200m Sprint	Brian	21.80	22.03
200m Sprint	Carlos	21.80	22.03
200m Sprint	Alice	22.50	22.03

SQL NEW QUESTION 1

SQL Question: Days Between Customer Orders

Marks: 20

Database: CUSTOMERS_ORDERS_DB

Schema Details

1. Customers Table

Column Name	Data Type	Description
customer_id	INT (PK)	Unique customer ID
customer_name	VARCHAR(100)	Customer name

2. Orders Table

Column Name	Data Type	Description
order_id	INT (PK)	Unique order ID
customer_id	INT (FK)	Customer who placed the order
order_date	DATE	Date when the order was placed
amount	DECIMAL(10,2)	Order amount

Sample Data

Customers

customer_id	customer_name
1	Aarav
2	Diya
3	Kabir

Orders

order_id	customer_id	order_date	amount
501	1	2025-08-02	800.00
506	1	2025-08-07	1200.00
514	1	2025-08-19	950.00
602	2	2025-08-04	650.00
609	2	2025-08-13	1050.00
701	3	2025-08-06	1500.00
708	3	2025-08-16	1100.00

Problem Statement

For each order placed by a customer, calculate the **number of days that have passed since the customer's immediately previous order**.

Use the **LAG() window function** to retrieve the previous order date for the same customer.

Requirements

- Process the orders separately for each customer using `customer_id`.
- Use `LAG()` to retrieve the immediately previous `order_date` for each customer.
- Determine the sequence of orders using `order_date` and then `order_id`.
- Calculate the number of days between the current order date and the previous order date.

- For the first order of each customer, days_since_previous_order should be NULL.
- Sort the final result by:
 1. customer_name
 2. order_date
 3. order_id

Required Output Columns

- customer_name
- order_id
- order_date
- days_since_previous_order

Expected Output

customer_name	order_id	order_date	days_since_previous_order
Aditi	101	2025-11-07T00:00:00.000Z	NULL
Aditi	102	2025-11-11T00:00:00.000Z	4
Aditi	105	2025-11-15T00:00:00.000Z	4
Bharat	201	2025-11-14T00:00:00.000Z	NULL

ANSWERS – SQL NEW QUESTION 2

SQL Question: Event Average Finish Time

Database: ATHLETICS_RESULTS_DB

Answer

```
SELECT
    e.event_name,
    a.athlete_name,
    r.finish_time,
    ROUND(
        AVG(r.finish_time) OVER (
            PARTITION BY r.event_id
        ), 2
    ) AS avg_event_time
FROM results r
JOIN events e
    ON r.event_id = e.event_id
JOIN athletes a
    ON r.athlete_id = a.athlete_id
WHERE r.status = 'Recorded'
ORDER BY
    e.event_name,
    r.finish_time,
    a.athlete_name;
```

Expected Output

event_name	athlete_name	finish_time	avg_event_time
100m Sprint	Brian	10.95	11.42
100m Sprint	Alice	11.20	11.42
100m Sprint	Carlos	12.10	11.42
200m Sprint	Brian	21.80	22.03
200m Sprint	Carlos	21.80	22.03
200m Sprint	Alice	22.50	22.03

ANSWERS – SQL NEW QUESTION 1

SQL Question: Days Between Customer Orders

Database: CUSTOMERS_ORDERS_DB

Answer

```
SELECT
  c.customer_name,
  o.order_id,
  o.order_date,
  DATEDIFF(
    o.order_date,
    LAG(o.order_date) OVER (
      PARTITION BY o.customer_id
      ORDER BY o.order_date, o.order_id
    )
  ) AS days_since_previous_order
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
ORDER BY
  c.customer_name,
  o.order_date,
  o.order_id;
```

Expected Output shown in the PDF

customer_name	order_id	order_date	days_since_previous_order
Aditi	101	2025-11-07T00:00:00.000Z	NULL
Aditi	102	2025-11-11T00:00:00.000Z	4
Aditi	105	2025-11-15T00:00:00.000Z	4
Bharat	201	2025-11-14T00:00:00.000Z	NULL